

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovreste consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

if  $n \leq 0$  then
  | return 1
else
  |  $k = 0$ 
  | for  $i = 1, \dots, n/2$  do
  |   |  $k = k + 1$ 
  | return MYSTERY1( $k$ ) + MYSTERY1( $k$ ) + MYSTERY1( $k$ ) + MYSTERY1( $k$ ) + MYSTERY2( $k$ ) +  $k^2$ 

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  | return 1
else
  |  $k = n$ 
  | for  $i = 1, \dots, n/2$  do
  |   |  $k = k - 1$ 
  | return MYSTERY2( $k$ ) + MYSTERY2( $k$ ) + MYSTERY2( $k$ ) + MYSTERY2( $k$ ) +  $k^2$ 

```

Soluzione. Analizziamo prima il costo di MYSTERY2. Tale funzione esegue un ciclo $n/2$ volte in cui ad ogni iterazione decrementa di 1 la variabile k , inizializzata ad n . Il ciclo ha un costo $\Theta(n)$ ed al termine di questo la variabile k ha valore $n/2$. Quindi la funzione MYSTERY2 esegue quattro chiamate ricorsive con input $k = n/2$. L'equazione di ricorrenza che descrive il costo di MYSTERY2 è quindi

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ 4T'(n/2) + n & n > 1 \end{cases}$$

Tale equazione di ricorrenza può essere risolta con il Master Theorem

$$\alpha = \log_2 4 = 2 > 1 = \beta \Rightarrow T'(n) = \Theta(n^\alpha) = \Theta(n^2)$$

Anche la funzione MYSTERY1 esegue un ciclo $n/2$ volte ma ad ogni iterazione incrementa di 1 la variabile k , inizializzata a zero. Come per MYSTERY2, il ciclo ha un costo $\Theta(n)$ ed al termine di questo la variabile k ha valore $n/2$. D'altra parte la funzione MYSTERY1 ha una chiamata alla

funzione MYSTERY2 con input $k = n/2$ che ha costo $\Theta(n^2/4) = \Theta(n^2)$. Tale costo sovrasta il costo del ciclo for. Per concludere, MYSTERY1 esegue quattro chiamate ricorsive con input $k = n/2$. L'equazione di ricorrenza che descrive il costo di MYSTERY1 è quindi

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 4T(n/2) + n^2 & n > 1 \end{cases}$$

Tale equazione di ricorrenza può essere risolta con il Master Theorem

$$\alpha = \log_2 4 = 2 = \beta \Rightarrow T(n) = \Theta(n^\alpha \log n) = \Theta(n^2 \log n)$$

2. Sia A un array contenente n interi. Diciamo che l'array A è k -disordinato se esistono k indici $I = \{i_1, \dots, i_k\} \subseteq \{1, \dots, n\}$ tali per cui

$$A_{i-1} \leq A_i \text{ se e solo se } i \notin I$$

In pratica A è totalmente ordinato tranne che per k elementi. Analizzare il costo per ordinare un array k -disordinato A di dimensione n con il *mergesort* e l'*insertionsort* quando:

- a) $k = O(1)$
- b) $k = \Theta(\log n)$
- c) $k = \Theta(\sqrt{n})$

Per ognuno dei tre casi indicare quale dei due algoritmi di ordinamento è più conveniente in termini di costo computazionale.

Soluzione. Il costo dell'algoritmo *mergesort* non dipende dal numero k di elementi non ordinati nell'array ma unicamente dalla dimensione dell'array. In tutti e tre i casi sarà quindi $\Theta(n \log n)$. Il costo dell'algoritmo *insertionsort* dipende invece dal numero di elementi disordinati. Ricordiamo che l'*insertionsort* sposta un elemento A_i di una posizione a sinistra (i.e. lo scambia con A_{i-1}) se e solo se $A_{i-1} > A_i$. Nel caso peggiore un elemento è scambiato con quello precedente per n volte. Dato che un array k -disordinato ha solo k elementi che richiedono di essere riposizionati, nel caso pessimo il costo di *insertionsort* sarà $O(kn)$.

- a) In questo caso il costo di *insertionsort* è lineare $O(n)$ nel caso pessimo ed è quindi più conveniente del *mergesort* che ha un costo pseudolineare $\Theta(n \log n)$
 - b) In questo caso il costo di *insertionsort* è pseudolineare $O(n \log n)$ nel caso pessimo ed è quindi equivalente al *mergesort* nel caso pessimo, ma potrebbe essere in pratica più efficiente in situazioni particolari che ricadono nel caso ottimo.
 - c) In questo caso il costo di *insertionsort* è polinomiale $O(n^{3/2})$ e risulta essere meno conveniente del *mergesort*.
3. Dato un array $A[1..n]$ di numeri ordinati in modo non decrescente ed un numero v , progettare un algoritmo dal costo computazionale $O(\log n)$ che restituisce il numero di occorrenze del numero v nell'array $A[1..n]$.

Soluzione.

Si può procedere con un algoritmo ricorsivo di tipo divide-et-impera che evita di fare il passo ricorsivo se è possibile verificare immediatamente che gli elementi del sottovettore $A[i..j]$ da considerare sono tutti uguali oppure tutti diversi rispetto a v . Visto che il vettore è ordinato in modo non decrescente, per verificare se tutti gli elementi in $A[i..j]$ sono uguali a v è sufficiente controllare se $A[i] = v$ e $A[j] = v$; per controllare se tutti sono diversi è sufficiente controllare se $A[i] > v$ (quindi tutti gli elementi dell'array sono strettamente maggiori di v) oppure $A[j] < v$ (quindi tutti gli elementi dell'array sono strettamente minori di v).

Tale algoritmo (si veda Algoritmo 2) viene inizialmente invocato con $i = 1$ e $j = n$. Si consideri ora un certo livello di annidamento delle chiamate ricorsive: visto che l'array è ordinato, al più due istanze delle funzioni a tale livello effettueranno le 2 chiamate ricorsive (le istanze relative a sottovettori $A[i..j]$ tali che $A[i] < v$ AND $A[j] \geq v$ oppure $A[i] \leq v$ AND $A[j] > v$). Quindi ad ogni livello al più 4 istanze possono essere invocate. Il numero massimo di livelli di annidamento è logaritmico rispetto a n in quanto la lunghezza dei sottovettori si dimezza ad ogni passaggio di livello. Il numero totale di istanze eseguite è quindi logaritmico rispetto a n . Visto che ogni istanza esegue un numero costante di operazioni elementari, si ottiene il costo computazionale $O(\log n)$.

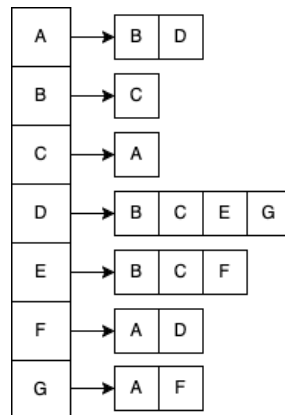
Algorithm 2: $\text{CONTAISTANZE}(\text{Number } A[1..n], \text{Integer } i, \text{Integer } j, \text{Number } v) \rightarrow \text{Integer}$

```

if  $i > j$  then
  | return 0
else if  $A[i] = v$  and  $A[j] = v$  then
  | return  $j - i + 1$ 
else if  $A[i] > v$  or  $A[j] < v$  then
  | return 0
else
  |  $\text{Integer } m = \text{Floor}((i + j)/2)$ 
  | return  $\text{CONTAISTANZE}(A, i, m, v) + \text{CONTAISTANZE}(A, m+1, j, v)$ 

```

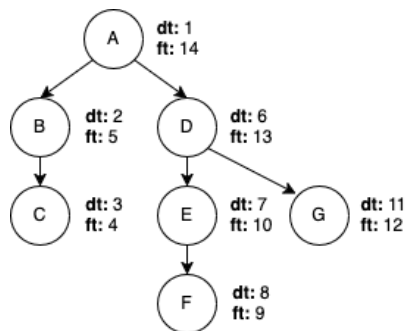
4. Si consideri il grafo orientato rappresentato tramite liste di adiacenza come da seguente diagramma:



Si disegni l'albero di visita che viene restituito dall'esecuzione della visita DFS a partire dal vertice A. A fianco di ogni vertice visitato, riportare anche i corrispondenti *discovery time* e *finish time*.

Soluzione.

La visita DFS a partire dal vertice A, visita inizialmente A, poi B (primo elemento della lista di adiacenze di A), poi C. A questo punto la visita di C e B si chiude in quanto non è possibile raggiungere altri nuovi vertici. Una volta tornati al vertice di partenza, la visita esplora D (secondo elemento della lista di adiacenze di A), poi E ed F. Si chiude la visita di F ed E, e tornati a D si procede con la visita di G. Successivamente si chiudono le visite di tutti i nodi ancora aperti. L'albero di visita avrà quindi la seguente struttura:



Si noti che a fianco di ogni vertice vengono riportati i corrispondenti *discovery time* (dt) e *finish time* (ft).